EXPERIMENT 12 – CNN ARCHITECTURE AND HYPERPARAMETER COMPARISON

1. Title

CNN Architecture and Hyperparameter Comparison

2. Aim

To compare different CNN architectures and hyperparameters by varying the number of convolutional layers, filter sizes, pooling methods, batch normalisation, learning rates and batch sizes. To evaluate the models based on accuracy, parameter count and training time.

3. Mathematical Formulae

1. Convolution Operation

$$Y(i,j) = \sum_m \sum_n X(i+m, j+n) K(m,n) + b$$

2. ReLU Activation Function

$$f(x) = \max(0, x)$$

3. Max Pooling

$$P(i,j) = \max_{(m,n) \in R} X(i+m, j+n)$$

4. Average Pooling

$$P(i,j) = \frac{1}{|R|} \sum_{(m,n) \in R} X(i+m, j+n)$$

5. Categorical Cross-Entropy Loss

$$L = -\sum_{i=1}^{C} y_i \log(\hat{y}_i)$$

6. Classification Accuracy

$$Accuracy = \frac{Correct\ Predictions}{Total\ Predictions} \times 100$$

7. Number of Convolution Layer Parameters

$$Parameters = (K_h \times K_w \times C_{in} \times C_{out}) + C_{out}$$

Where:

- K_h = Kernel height
- K_w = Kernel width
- C_{in} = Number of input channels
- C_{out} = Number of output filters

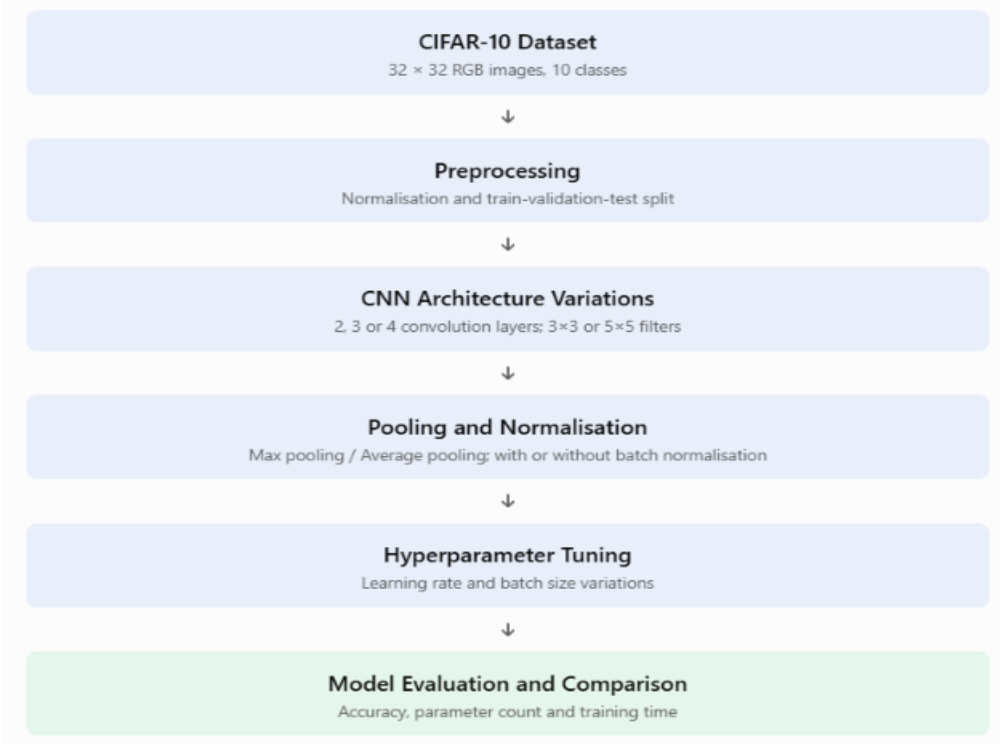
8. Batch Normalisation

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y = \gamma \hat{x} + \beta$$

Where μ is the mean, σ^2 is the variance, and γ and β are learnable parameters.

4. System Architecture



5. Algorithm

1. Import the required libraries such as TensorFlow, NumPy, Matplotlib and Pandas.
2. Load the CIFAR-10 dataset.
3. Normalise the image pixel values between 0 and 1.
4. Divide the dataset into training, validation and testing sets.
5. Define a CNN model-building function with configurable parameters.
6. Vary the number of convolutional layers between 2, 3 and 4.
7. Experiment with filter sizes of 3×3 and 5×5 and different filter counts.
8. Compare max pooling and average pooling.
9. Train models with and without batch normalisation.
10. Vary learning rates and batch sizes.
11. Record the accuracy, number of parameters and training time for each model.
12. Compare the experimental results using tables and graphs.

6. Python Program

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# 1. Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = (
   tf.keras.datasets.cifar10.load_data()
)

# 2. Normalise images
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

# 3. Split training data into training and validation
x_val = x_train[45000:]
```

```
y_val = y_train[45000:]

x_train = x_train[:45000]
y_train = y_train[:45000]

classes = [
  "Airplane", "Automobile", "Bird", "Cat", "Deer",
  "Dog", "Frog", "Horse", "Ship", "Truck"
]

# 4. Build CNN model
model = tf.keras.Sequential([

  tf.keras.layers.Input(shape=(32, 32, 3)),

  tf.keras.layers.Conv2D(
    32, (3, 3), activation="relu",
    padding="same", name="conv1"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Conv2D(
    64, (3, 3), activation="relu",
    padding="same"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Conv2D(
    128, (3, 3), activation="relu",
    padding="same"
  ),
  tf.keras.layers.MaxPooling2D((2, 2)),

  tf.keras.layers.Flatten(),
```

```
    tf.keras.layers.Dense(128, activation="relu"),

    tf.keras.layers.Dropout(0.3),

    tf.keras.layers.Dense(10, activation="softmax")
])

# 5. Compile model
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

model.summary()

# 6. Train model
history = model.fit(
    x_train,
    y_train,
    validation_data=(x_val, y_val),
    epochs=10,
    batch_size=32
)

# 7. Plot accuracy
plt.figure(figsize=(8, 5))

plt.plot(history.history["accuracy"], label="Training Accuracy")
plt.plot(history.history["val_accuracy"], label="Validation Accuracy")

plt.title("CNN Accuracy")
plt.xlabel("Epochs")
```

```
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# 8. Plot loss
plt.figure(figsize=(8, 5))

plt.plot(history.history["loss"], label="Training Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")

plt.title("CNN Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# 9. Evaluate model
test_loss, test_accuracy = model.evaluate(x_test, y_test)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)

# 10. Confusion matrix
predictions = model.predict(x_test)

y_pred = np.argmax(predictions, axis=1)

y_true = y_test.flatten()

cm = confusion_matrix(y_true, y_pred)

display = ConfusionMatrixDisplay(
   confusion_matrix=cm,
   display_labels=classes
```

```
)

display.plot(cmap="Blues", xticks_rotation=45)

plt.title("Confusion Matrix")
plt.show()

# 11. Display misclassified images
misclassified = np.where(y_true != y_pred)[0]

plt.figure(figsize=(12, 6))

for i, index in enumerate(misclassified[:6]):

  plt.subplot(2, 3, i + 1)

  plt.imshow(x_test[index])

  plt.title(
    "Actual: " + classes[y_true[index]] +
    "\nPredicted: " + classes[y_pred[index]]
  )

  plt.axis("off")

plt.tight_layout()
plt.show()

# 12. Visualise feature maps
feature_model = tf.keras.Model(
  inputs=model.inputs,
  outputs=model.get_layer("conv1").output
)
```

```
feature_maps = feature_model.predict(
  x_test[:1]
)

plt.figure(figsize=(10, 10))

for i in range(16):

  plt.subplot(4, 4, i + 1)

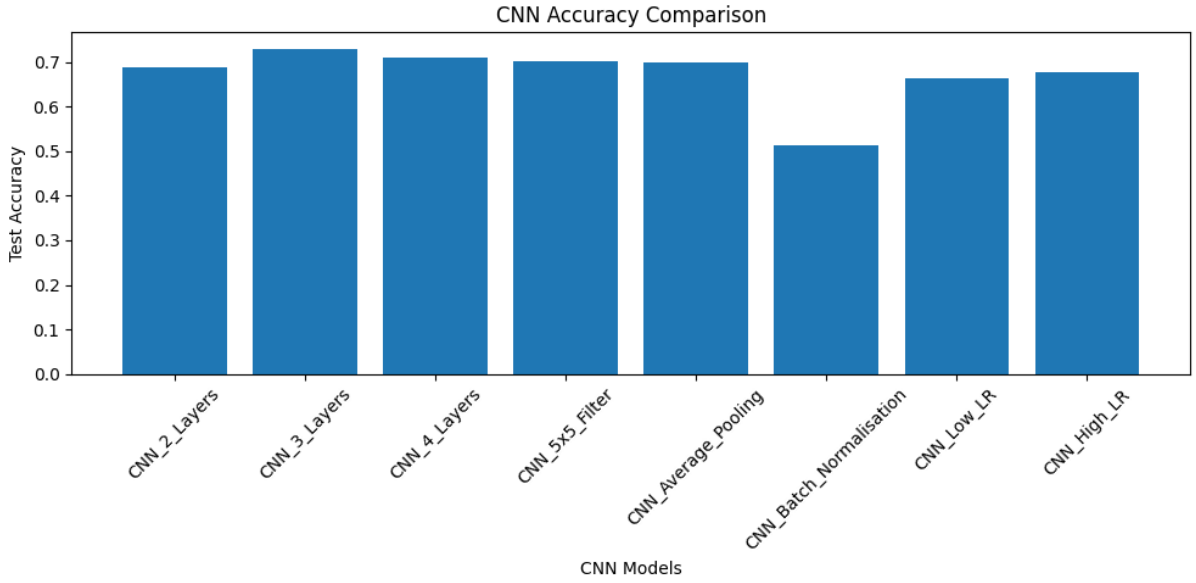
  plt.imshow(
    feature_maps[0, :, :, i],
    cmap="viridis"
  )

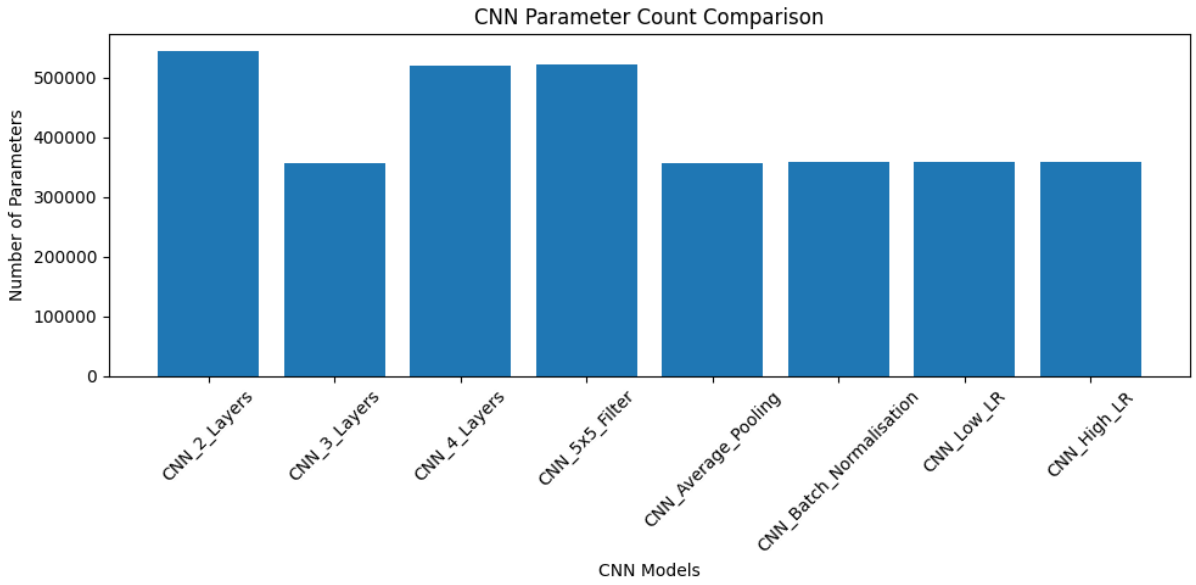
  plt.axis("off")

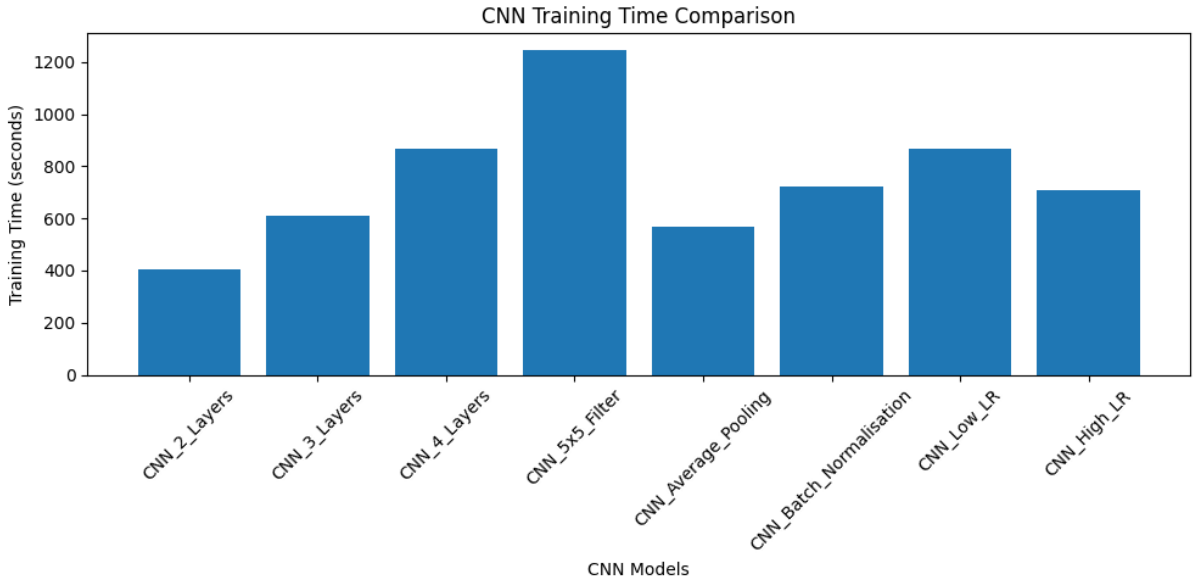
plt.suptitle("CNN Feature Maps")
plt.show()
```

7. Output

```
CNN MODEL COMPARISON
                    Model  Accuracy  Parameters  Training Time
             CNN_2_Layers    0.6884      545098     406.092305
             CNN_3_Layers    0.7306      356810     611.212038
             CNN_4_Layers    0.7096      520906     867.531389
          CNN_5x5_Filter     0.7031      522186    1246.887400
     CNN_Average_Pooling     0.6999      356810     568.178356
CNN_Batch_Normalisation      0.5130      357706     720.752492
                CNN_Low_LR   0.6652      357706     866.757989
               CNN_High_LR   0.6778      357706     708.726247
```


CNN Accuracy Comparison
Test Accuracy
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
CNN_2_Layers
CNN_3_Layers
CNN_4_Layers
CNN_5x5_Filter
CNN_Average_Pooling
CNN_Batch_Normalisation
CNN_Low_LR
CNN_High_LR
CNN Models


CNN Parameter Count Comparison
Number of Parameters
0
100000
200000
300000
400000
500000
CNN_2_Layers
CNN_3_Layers
CNN_4_Layers
CNN_5x5_Filter
CNN_Average_Pooling
CNN_Batch_Normalisation
CNN_Low_LR
CNN_High_LR
CNN Models


CNN Training Time Comparison
Training Time (seconds)
0
200
400
600
800
1000
1200
CNN_2_Layers
CNN_3_Layers
CNN_4_Layers
CNN_5x5_Filter
CNN_Average_Pooling
CNN_Batch_Normalisation
CNN_Low_LR
CNN_High_LR
CNN Models

8. Result

Thus, different CNN architectures and hyperparameters were implemented and compared successfully using the CIFAR-10 dataset. The models were evaluated based on classification accuracy, number of trainable parameters and training time to analyse the effect of architectural and hyperparameter variations.